

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-a9c7a5c58544cf74f.jsonl`  
- SHA-256: `928f2feb567c755b8782f7ae36647ccd176b62e7e01808512e2eff92a1459ee9`  
- Source modified: `2026-07-06T09:12:55+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_3512746c-54e`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:09:16.746Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).
It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +
an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY:
command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate),
tests/test_command_service.py (10 tests).
Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel
creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff
 and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur /
 pre-existing. PLAUSIBLE if unsettled. Corrected severity.
 FINDING (tests_ux): /watch cannot resolve a channel by name/username for real sources (only numeric id works)
 severity medium | src/tg_video_filter/command_service.py:402
 summary: ADR 0009 line 64 specifies '/watch <#name|id>' (resolve by channel name/username OR id), but _resolve_source only looks up provider_source_id, and every production code path stores that as the numeric channel id (str(channel_id)) -- so /watch @username and /watch by title always fail for real channels.
 scenario: A user sees a channel titled 'Classical Music' in /channels and types '/watch Classical Music' or '/watch @classicalmusic'. _resolve_source (command_service.py:402-416) strips '@' and queries get_source_by_provider_id with the text as provider_source_id, then retries with an '@' prefix. Real sources are stored with provider_source_id = str(channel_id) (a numeric string like -1001234567890) at db.py:1399 (upsert_channel) and db.py:1576 (insert_video) -- never as a username or title. Both lookups miss, and the user gets '? Source not found', even though the channel is right there in /channels. Only pasting the raw numeric id works. The test_watch_by_username test passes only because it manually seeds a source whose provider_source_id is literally '@mychan' (test line 302) -- a shape no production path ever creates.
 evidence: grep for provider_source_id= in src shows the only assignments are the row reader (db.py:848) and the two str(channel_id) writers (db.py:1399, 1576). _resolve_source never queries the title column. ADR line 64 quotes '/watch <#name|id>'.
 rec: Resolve by title/username in addition to numeric id: query sources by title (case-insensitive) and, if usernames are to be supported, persist the channel username on the source row so it can be matched. At minimum, document that only numeric-id resolution is supported and correct the misleading test that implies username resolution works.

2. assistant (2026-07-06T09:09:19.156Z)

I'll verify this finding by reading the actual code and ADR.

3. user (2026-07-06T09:09:20.173Z)

```

380         lines = ["? **All sources**", ""]
381         for s in sources:
382             icon = "?" if s.watched else ""
383             label = s.title or s.provider_source_id
384             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385         return CommandResult("\n".join(lines))
386
387     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
388         """List only watched sources."""
389         sources = await db.list_account_sources(conn, user_id, watched_only=True)
390         if not sources:
391             return CommandResult(
392                 "? **No watched sources.**\n\n"
393                 "Use `/channels` to see available sources, then `/watch <id>` "
394                 "to start scanning."
395             )
396         lines = ["? **Watched sources**", ""]
397         for s in sources:
398             label = s.title or s.provider_source_id
399             lines.append(f"? `{s.provider_source_id}` ? {label}")
400         return CommandResult("\n".join(lines))
401
402     async def _resolve_source(
403         self, conn: aiomysqlite.Connection, user_id: int, target: str
404     ) -> Source | None:
405         """Resolve a user-provided source identifier to a ``Source`` row.
406
407         Accepts ``@username`` or a numeric channel id string. The ``@`` prefix

```

```

408         is stripped before lookup since Telegram channel ids are numeric.
409         ""
410         target = target.lstrip("@")
411         # Try by provider-native id first (the common case).
412         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
413         if src is not None:
414             return src
415         # Fall back: try with @ prefix in the target column (just in case).
416         return await db.get_source_by_provider_id(conn, user_id, "telegram",
f"@{target}")
417
418     # -- internal -----
419
420     async def _update(
421         self,
422         conn: aiosqlite.Connection,
423         user_id: int,
424         current: FilterConfig,
425         max_size_mb: int | None = None,
426         max_duration_s: int | None = None,
427         min_duration_s: int | None = None,
428         target_height: int | None = None,
429         exclude_gif: bool | None = None,
430         exclude_round: bool | None = None,
431         exclude_nosound: bool | None = None,
432     ) -> FilterConfig:
433         """Apply partial updates to the account's config, persist, and return."""
434         updates: dict[str, object] = {}
435         if max_size_mb is not None:
436             updates["max_size_mb"] = max_size_mb
437         if max_duration_s is not None:
438             updates["max_duration_s"] = max_duration_s
439         if min_duration_s is not None:

```

4. user (2026-07-06T09:09:20.651Z)

```

1     # ADR 0009: Selective channel scanning and multiplexed routing
2
3     - **Status**: Accepted
4     - **Date**: 2026-07-04
5     - **Decision owner**: project owner
6
7     ## Context
8
9     Today the userbot scans every channel/group the account is a member of.
10    `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11    filter, and `on_new_message` processes every video regardless of source. The
12    `channels` table exists but is populated lazily as a record of seen channels;
13    it is not used as a filter.
14
15    The product vision is a multiplexing video re-distribution system:
16
17    - A small set of high-quality source channels (e.g. spiritual, classical
18      music) are scanned for incoming videos.
19    - Matches are routed to themed destination broadcast channels (e.g. a
20      Reggae channel, a Rap channel) that subscribers opt into.
21    - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23    This requires two capabilities the current architecture does not have:
24
25    1. Selective scanning ? choose which source channels are watched, rather
26      than ingesting all of them.
27    2. Per-source ? per-destination routing ? a match from source channel X is
28      delivered to a specific destination channel Y, instead of the single global
29      `delivery_target` (ADR 0005).

```

```

30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.
89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.

```

```

95 - Each capability is additive and independently deployable; none disturbs the
96 existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98 file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100 `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105 may be a member of a *destination* broadcast channel and re-see its own
106 output, producing an infinite forward loop. The contract is: **a channel
107 is either a source or a destination, never both.** This is enforced in
108 `on_new_message` by excluding any channel that is a configured destination
109 (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110 user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111 this; the invariant is stated here so the handler has a contract to code
112 against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114 filters only and a single implicit route (`source ? global target`), then
115 layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order
136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138 `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140 switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142 `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150 watched set is per-user and mutable at runtime; re-registering Telethon
151 handlers on every `/watch` change is brittle. An in-handler lookup against
152 the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of
154 scope for v1. Routing is user-defined by source channel; automatic topic
155 detection could be a future enhancement but is not required to deliver the
156 multiplexing UX.
157
158 ## References
159

```

```

160 - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161 - ADR 0005 (per-user delivery target) ? the single-target delivery this
162 generalises into per-route targets.
163 - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164 the original `document_id`.
165 - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166 `/unwatch`, and routing commands to public users.
167

```

5. assistant (2026-07-06T09:09:24.680Z)

Now let me verify the key claims. Let me check `get\_source\_by\_provider\_id`, the `Source` schema (does it have a title/username column), and how real sources are stored.

6. user (2026-07-06T09:09:26.313Z)

```

src\tg_video_filter\db.py:578:         AND s.provider_source_id = CAST(v.channel_id AS TEXT)
src\tg_video_filter\db.py:752:         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
src\tg_video_filter\db.py:848:         provider_source_id=row["provider_source_id"],
src\tg_video_filter\db.py:1134:         WHERE account_id = ? AND provider = ? AND
src\tg_video_filter\db.py:1149:         WHERE account_id = ? AND provider = ? AND
src\tg_video_filter\db.py:1399:         provider_source_id=str(channel.channel_id),
src\tg_video_filter\db.py:1576:         provider_source_id=str(video.channel_id),

```

7. user (2026-07-06T09:09:27.574Z)

```

src\tg_video_filter\db.py:1104:async def upsert_source(conn: aiosqlite.Connection, source:
Source) -> Source:
src\tg_video_filter\db.py:1142:async def get_source_by_provider_id(
src\tg_video_filter\db.py:1387:async def upsert_channel(conn: aiosqlite.Connection, channel:
Channel) -> Channel:

```

8. assistant (2026-07-06T09:09:29.930Z)

Let me look at the Source dataclass/schema (title, username columns) and the surrounding code at those key lines.

9. user (2026-07-06T09:09:31.039Z)

```

820     """Project an (account, optional session) pair onto the legacy ``User`` view.
821
822     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
823     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
824     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
825     "list enabled users" semantics exclude it ? matching the structural rule
826     that bot-only users are not runnable Telethon clients.
827     """
828     if session is None:
829         return User(
830             id=account.telegram_id if account.telegram_id is not None else account.id,
831             phone=None,
832             enabled=False,
833             created_at=account.created_at,
834         )
835     return User(
836         id=account.telegram_id if account.telegram_id is not None else account.id,
837         phone=session.phone,
838         enabled=session.enabled,
839         created_at=session.created_at,
840     )
841
842

```

```

843 def _row_to_source(row: aiosqlite.Row) -> Source:
844     return Source(
845         id=row["id"],
846         account_id=row["account_id"],
847         provider=row["provider"],
848         provider_source_id=row["provider_source_id"],
849         title=row["title"],
850         kind=row["kind"],
851         watched=bool(row["watched"]),
852         seen_first_at=row["seen_first_at"],
853         last_seen_at=row["last_seen_at"],
854     )
855
856
857 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
858     return ContentItem(
859         id=row["id"],

```

10. user (2026-07-06T09:09:31.946Z)

```

1104 async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1105     """Insert a source row, or update title/watched on conflict.
1106
1107     The conflict key is `` (account_id, provider, provider_source_id)`` per the
1108     ADR 0011 schema. Returns the persisted row.
1109     """
1110     await conn.execute(
1111         """INSERT INTO sources
1112             (account_id, provider, provider_source_id, title, kind, watched,
1113              seen_first_at, last_seen_at)
1114             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                 title=excluded.title,
1117                 watched=excluded.watched,
1118                 last_seen_at=excluded.last_seen_at""",
1119         (
1120             source.account_id,
1121             source.provider,
1122             source.provider_source_id,
1123             source.title,
1124             source.kind,
1125             int(source.watched),
1126             source.seen_first_at.isoformat(),
1127             (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128         ),
1129     )
1130     await conn.commit()
1131     conn.row_factory = aiosqlite.Row
1132     async with conn.execute(
1133         """SELECT * FROM sources
1134             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135         (source.account_id, source.provider, source.provider_source_id),
1136     ) as cur:
1137         row = await cur.fetchone()
1138         assert row is not None # just upserted by the conflict key
1139         return _row_to_source(row)
1140
1141
1142 async def get_source_by_provider_id(
1143     conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
1144 ) -> Source | None:
1145     """Resolve a source by its provider-native id."""
1146     conn.row_factory = aiosqlite.Row
1147     async with conn.execute(
1148         """SELECT * FROM sources

```

```

1149         WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1150         (account_id, provider, provider_source_id),
1151     ) as cur:
1152         row = await cur.fetchone()
1153     return _row_to_source(row) if row else None
1154
1155
1156 # --- ContentItem repos (ADR 0011) -----
1157
1158
1159 async def insert_content_item(conn: aioredis.Connection, item: ContentItem) ->
ContentItem:
1160     """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
1161
1162     The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
1163     unique constraints guard against duplicates:
1164     - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1165     - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1166     """
1167     cur = await conn.execute(
1168         """INSERT INTO content_items
1169             (account_id, source_id, provider, provider_message_id, content_type,
1170             text, link_url, media_ref, document_id, duration_s, width, height,
1171             size_bytes, mime_type, dedup_key, matched, seen_at)
1172             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
1173         (

```

11. user (2026-07-06T09:09:32.613Z)

```

1387 async def upsert_channel(conn: aioredis.Connection, channel: Channel) -> Channel:
1388     """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389
1390     ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391     reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392     New code should call :func:`upsert_source` directly.
1393     """
1394     src = await upsert_source(
1395         conn,
1396         Source(
1397             account_id=channel.user_id,
1398             provider="telegram",
1399             provider_source_id=str(channel.channel_id),
1400             title=channel.title,
1401             kind="channel",
1402             watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403             seen_first_at=channel.seen_first_at,
1404         ),
1405     )
1406     return _source_to_channel(src)
1407
1408
1409 async def get_channel(conn: aioredis.Connection, user_id: int, channel_id: int) ->
Channel | None:
1410     """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412     return _source_to_channel(src) if src else None
1413
1414
1415 def _source_to_channel(src: Source) -> Channel:
1416     """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
```

12. assistant (2026-07-06T09:09:37.919Z)

The finding is well-supported. Let me confirm the two facts:

1. `\_resolve\_source` (command_service.py:410-416) strips `@` and only queries `get\_source\_by\_provider\_id`, which matches only the `provider\_source\_id` column (db.py:1148-1150). It never queries the `title` column.
2. Real sources are stored with `provider\_source\_id = str(channel\_id)` ? numeric ? at db.py:1399 (upsert_channel) and db.py:1576 (insert_video). The `title` column holds the human name.

Let me verify the test the finding cites to confirm it seeds an artificial shape, and check the `/channels` upsert path to confirm no real path stores a username as provider_source_id.

13. user (2026-07-06T09:09:38.936Z)

```
280         await _seed_source(conn, USER_ID, "-1001", "Ch A", watched=True)
281         await _seed_source(conn, USER_ID, "-1002", "Ch B", watched=False)
282         result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
283         assert "Ch A" in result.reply_text
284         assert "Ch B" not in result.reply_text
285
286
287     async def test_watchlist_empty_shows_help(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
288         result = await svc.handle(USER_ID, "/watchlist", conn, FilterConfig())
289         assert "No watched sources" in result.reply_text
290
291
292     async def test_watch_by_numeric_id(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
293         await _seed_source(conn, USER_ID, "-1001", "Target", watched=False)
294         result = await svc.handle(USER_ID, "/watch -1001", conn, FilterConfig())
295         assert "Now watching" in result.reply_text
296         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
297         assert src is not None
298         assert src.watched is True
299
300
301     async def test_watch_by_username(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
302         await _seed_source(conn, USER_ID, "@mychan", "My Channel", watched=False)
303         result = await svc.handle(USER_ID, "/watch @mychan", conn, FilterConfig())
304         assert "Now watching" in result.reply_text
305         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "@mychan")
306         assert src is not None
307         assert src.watched is True
308
309
310     async def test_unwatch(conn: aiomysqlite.Connection, svc: CommandService) -> None:
311         await _seed_source(conn, USER_ID, "-1001", "Target", watched=True)
312         result = await svc.handle(USER_ID, "/unwatch -1001", conn, FilterConfig())
313         assert "Stopped watching" in result.reply_text
314         src = await db.get_source_by_provider_id(conn, USER_ID, "telegram", "-1001")
315         assert src is not None
316         assert src.watched is False
317
318
319     async def test_watch_unknown_source_shows_error(
```

14. user (2026-07-06T09:09:40.459Z)

```
320:         return await self._handle_watch(conn, user_id, m.group(1).strip(), current_cfg)
327:         return await self._handle_channels(conn, user_id)
341:     async def _handle_watch(
345:         src = await self._resolve_source(conn, user_id, target)
361:         src = await self._resolve_source(conn, user_id, target)
370:     async def _handle_channels(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
```

```
402:     async def _resolve_source(
```

15. assistant (2026-07-06T09:09:43.360Z)

Let me check `\_handle\_channels` to confirm whether it upserts from `get\_dialogs()` (ADR line 52) ? and whether any username is ever persisted, which is relevant to the finding's scope.

16. user (2026-07-06T09:09:44.187Z)

```
370         async def _handle_channels(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
371             """List all known sources with watch status."""
372             sources = await db.list_account_sources(conn, user_id)
373             if not sources:
374                 return CommandResult(
375                     "? **No sources yet.**\n\n"
376                     "Sources are discovered when a new message arrives from a "
377                     "channel. Run `/backfill` to discover historical channels, "
378                     "or wait for new traffic."
379                 )
380             lines = ["? **All sources**", ""]
381             for s in sources:
382                 icon = "?" if s.watched else "?"
383                 label = s.title or s.provider_source_id
384                 lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
385             return CommandResult("\n".join(lines))
386
387         async def _handle_watchlist(self, conn: aiosqlite.Connection, user_id: int) ->
CommandResult:
388             """List only watched sources."""
389             sources = await db.list_account_sources(conn, user_id, watched_only=True)
390             if not sources:
391                 return CommandResult(
392                     "? **No watched sources.**\n\n"
393                     "Use `/channels` to see available sources, then `/watch <id>` "
394                     "to start scanning."
395                 )
396             lines = ["? **Watched sources**", ""]
397             for s in sources:
398                 label = s.title or s.provider_source_id
399                 lines.append(f"? `{s.provider_source_id}` ? {label}")
400             return CommandResult("\n".join(lines))
401
```

17. assistant (2026-07-06T09:09:52.275Z)

This confirms the finding thoroughly, and reveals it understates the situation. `/channels` displays sources with `label = s.title or s.provider\_source\_id` ? so users literally see the title (e.g. "Classical Music") in the listing, then naturally type `/watch Classical Music`, which cannot resolve. The finding's scenario is accurate.

Now the adversarial refutation attempt. Is there ANY production path that stores a username or title as `provider\_source\_id`? The grep shows only two writers: db.py:1399 (`str(channel.channel\_id)`) and db.py:1576 (`str(video.channel\_id)`) ? both numeric. `\_handle\_channels` doesn't upsert usernames. So `/watch @username` and `/watch <title>` genuinely always fail for real channels.

Let me run a quick repro to be certain the resolution fails on a numeric-stored source when queried by title.

18. user (2026-07-06T09:10:01.808Z)

Exit code 1

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 5, in <module>
    from tg_video_filter.config import FilterConfig
ImportError: cannot import name 'FilterConfig' from 'tg_video_filter.config'
(C:\Users\avidu\Projects\Telegram\src\tg_video_filter\config.py)
```

19. user (2026-07-06T09:10:06.950Z)

```
35:class Account(BaseModel):
123:class Source(BaseModel):
299:class FilterConfig(BaseModel):
21:from tg_video_filter.models import FilterConfig, Source
110:def format_config(cfg: FilterConfig) -> str:
111:    """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
```

20. user (2026-07-06T09:12:15.521Z)

Command running in background with ID: bz0k2aih. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bz0k2aih.output. You will be notified when it completes. To check interim output, use Read on that file path.

21. user (2026-07-06T09:12:24.071Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 24, in <module>
    asyncio.run(main())
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
    ~~~~~^~~~~~
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 11, in main
    acc = await db.upsert_account(conn, Account(telegram_id=42, kind="user"))
    ~~~~~^~~~~~
  File "C:\Users\avidu\Projects\Telegram\venv\Lib\site-packages\pydantic\main.py", line 263, in __init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Account
id
  Field required [type=missing, input_value={'telegram_id': 42, 'kind': 'user'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing
```

22. assistant (2026-07-06T09:12:25.943Z)

Let me check how tests seed sources ? the `\_seed\_source` helper in the test file.

23. user (2026-07-06T09:12:27.839Z)

```
26:async def conn() -> aiosqlite.Connection:
27:    """An in-memory SQLite connection with a seeded user and default config."""
28:    c = await aiosqlite.connect(":memory:")
29:    await db._apply_schema(c)
```

```

30-     await db.upsert_user(
31-         c,
32-         db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33-     )
34-     await db.save_filter_config(c, USER_ID, FilterConfig())
35-     yield c
36-     await c.close()
37-
38-
--
242:async def _seed_source(
243-    conn: aiosqlite.Connection,
244-    account_id: int,
245-    provider_source_id: str,
246-    title: str = "",
247-    watched: bool = False,
248-) -> Source:
249-    return await db.upsert_source(
250-        conn,
251-        Source(
252-            account_id=account_id,
253-            provider="telegram",
254-            provider_source_id=provider_source_id,

```

24. assistant (2026-07-06T09:12:30.175Z)

The tests use `db.upsert\_user`. Let me fix the repro to mirror that.

25. user (2026-07-06T09:12:36.929Z)

```

'/watch Classical Music' -> ? Source `Classical Music` not found. Use `/channels` to list
available sources.
'/watch @classicalmusic' -> ? Source `@classicalmusic` not found. Use `/channels` to list
available sources.
'/watch -1001234567890' -> ? Now watching **Classical Music** (`-1001234567890`).

```

26. assistant (2026-07-06T09:12:47.264Z)

Repro confirms the finding exactly. On a source stored the way every production path stores it (numeric `provider\_source\_id`, human title), `/watch Classical Music` and `/watch @classicalmusic` both return "Source not found", while only the raw numeric id works.

Verification summary:

- `\_resolve\_source` (command_service.py:410-416) only queries `provider\_source\_id`, never `title`. Confirmed by read.
- Real sources always have numeric `provider\_source\_id` (db.py:1399, 1576) and human name in `title`. Confirmed by grep ? only two writers, both `str(channel\_id)`.
- `/channels` displays `label = s.title` (line 383), so the user is shown the title, making `/watch <title>` the natural action ? which fails.
- ADR line 64 specifies `/watch <#name|id>` (name OR id). The code supports only id for real channels ? a genuine deviation from the accepted ADR.
- The `test\_watch\_by\_username` test (line 301-307) passes only because it seeds `provider\_source\_id="@mychan"`, a shape no production path creates ? the test is misleading, not evidence the feature works.

On severity: The feature is not broken on the live ingestion path (the gate itself works; numeric id resolves). It's the resolution UX that's half-broken ? the primary documented invocation form (`<#name>`) fails for all real channels, and `/channels` actively steers users toward the failing form. Users can still opt in via numeric id (visible in `/channels` output as the backticked value). That's a real, user-hit ADR-contract violation but with a working fallback, so medium is the honest severity ? matching the finding's own rating.

27. user (2026-07-06T09:12:55.092Z)

Structured output provided successfully